

Docker Basic Commands Assignment

Introduction

Docker is an open-source platform that automates the deployment, scaling, and management of applications using containerization. Understanding its basic commands is fundamental for anyone looking to leverage Docker for development, testing, or production environments. This assignment will explain the function and provide usage examples for five essential Docker commands: `docker ps`, `docker run`, `docker build`, `docker images`, and `docker logs`.

Basic Docker Commands

1. `docker ps`

Function: The `docker ps` command lists all running containers. It provides a quick overview of the containers currently active on your system, along with key information about each.

Usage Example:

```
docker ps
```

Explanation: This command displays a table with columns such as `CONTAINER ID`, `IMAGE`, `COMMAND`, `CREATED`, `STATUS`, `PORTS`, and `NAMES`. It helps in identifying running containers, checking their status, and managing them. To view all containers, including those that have stopped, you can use `docker ps -a`.

2. docker run

Function: The `docker run` command creates a new container and starts it. It is one of the most frequently used commands, allowing you to launch an application within a Docker container based on a specified image.

Usage Example:

```
docker run -d -p 80:80 --name my-web-server nginx
```

Explanation:

- `-d` : Runs the container in detached mode, meaning it runs in the background.
- `-p 80:80` : Maps port 80 of the host to port 80 of the container, allowing external access to the web server.
- `--name my-web-server` : Assigns a custom name (`my-web-server`) to the container for easier identification.
- `nginx` : Specifies the Docker image to use for creating the container. In this case, it's the official Nginx web server image.

3. docker build

Function: The `docker build` command builds Docker images from a Dockerfile and a “context”. A Dockerfile is a text document that contains all the commands a user could call on the command line to assemble an image.

Usage Example:

```
docker build -t my-custom-app .
```

Explanation:

- `-t my-custom-app` : Tags the newly built image with the name `my-custom-app` . This makes it easy to refer to the image later.

- `.` : Specifies the build context, which is the set of files at the specified PATH or URL. The Docker daemon sends the context recursively to the daemon. In this case, `.` refers to the current directory, where the Dockerfile is expected to be located.

4. docker images

Function: The `docker images` command lists all Docker images that are stored locally on your system. This command is useful for managing your local image cache and identifying images that can be used to create containers.

Usage Example:

```
docker images
```

Explanation: This command displays a table with information about each image, including its Repository, Tag, Image ID, Creation date, and Size. It helps in keeping track of downloaded and custom-built images.

5. docker logs

Function: The `docker logs` command is used to fetch the logs of a container. It allows you to view the standard output and standard error streams of a running or stopped container, which is invaluable for debugging and monitoring.

Usage Example:

```
docker logs my-web-server
```

Explanation: This command retrieves and displays the logs generated by the container named `my-web-server`. You can also use the Container ID instead of the name. The `-f` or `--follow` option can be used to stream new logs as they are generated (`docker logs -f my-web-server`).

Conclusion

These basic Docker commands form the foundation for interacting with Docker. Mastering them is crucial for anyone working with containerized applications, enabling efficient development, deployment, and troubleshooting workflows.